

DevStore E-Commerce Platform

Complete Technical Architecture & Implementation Learning Guide

1. Executive Summary & Overview

DevStore is a modern, high-performance full-stack e-commerce web application engineered with a **React 19 / Vite** frontend and a **Node.js / Express** backend. Designed for speed, security, and exceptional user experience, the system features a zero-setup persistent JSON storage engine, a dedicated **Enterprise Admin Gateway** at /admin, real-time background data synchronization, automatic 24-hour order shipping transitions, global address autocomplete, and custom pixel-perfect PDF invoice generation.

Layer	Technologies Used	Purpose & Highlights
Frontend Core	React 19, Vite, React Router v6	Single Page Application (SPA) with ultra-fast JSX rendering
UI & Styling	Tailwind CSS, Lucide Icons, Framer Motion	Glassmorphism, dark theme palette, smooth animations
Backend Core	Node.js, Express.js	RESTful API server with modular controllers & routes
Storage Engine	Dynamic JSON Storage (jsonDb.js)	Zero-database file collection storage in JSON format
Security & Auth	JWT, HTTP-Only Cookies, bcryptjs	Session tokens, password hashing & OTP registration
Documents & PDF	PDFKit	On-the-fly PDF creation for order financial breakdown

2. Deep-Dive Architecture & Core Features

- **Dedicated Admin Gateway (/admin):** Located directly at /admin, this portal provides restricted access exclusively to administrators. It features an isolated authentication card with 1-click Auto-fill credentials (admin@devstore.com / Admin123!), bypassing guest registration pathways.
- **Zero-Flicker Real-Time Background Sync:** To maintain real-time updates without distracting user interfaces, AdminPanel.jsx utilizes smart background polling. Initial page loads trigger loading skeletons only when data is missing, while subsequent polling updates React state silently without screen flashing.
- **Automated 24-Hour Shipping Transition:** All customer orders initially marked as processing automatically transition to shipped status after 24 hours. The backend checks timestamps dynamically (Date.now() - createdAt >= 86,400,000 ms) and updates storage immediately.
- **Enterprise PDF Invoice Generation:** Generates printable PDF receipts using PDFKit. Resolves financial calculation overlaps (Subtotal, Tax, Shipping, Discount, Grand Total) with dynamic vertical offsets and clean tabular alignment.

- **Address Autocomplete on Checkout:** Enhances user experience during checkout by providing dynamic location recommendations for street address, city, state, and country fields.

3. Directory Structure & File Map

Directory / File	Description
client/src/pages/AdminPanel.jsx	Dedicated Admin Console UI containing tabs for Dashboard, Products, Orders, Users, and Coupons.
client/src/context/AppContext.jsx	Global React State management handling user authentication, cart, wishlist, and toast notifications.
server/controllers/adminController.js	Backend handlers for dashboard statistics, product CRUD, 24-hour auto-shipping orders, and clean user listings.
server/controllers/authController.js	Authentication logic for registration, OTP generation, login validation, and JWT cookie management.
server/controllers/orderController.js	Order placement, checkout calculations, user order history, and PDF stream trigger endpoint.
server/services/invoiceService.js	Enterprise PDF generation engine built with PDFKit, layout coordinates, and formatting.
server/utils/jsonDb.js	Lightweight ORM abstraction mapping MongoDB-like methods (find, create, findByIdAndUpdate) to JSON files.
server/data/collections/	Persistent JSON database files (user.json, order.json, product.json).

4. How the Application Works (End-to-End Flow)

A. User Authentication & Security Flow:

1. User submits credentials on /login or /register.
2. Backend hashes password using bcryptjs and generates an OTP if registering.
3. Upon successful verification, an HTTP-Only JWT token is sent back in cookies.
4. Frontend AppContext authenticates session on boot via GET /api/auth/me.

B. Admin Panel & Management Flow:

1. Navigating to http://localhost:5173/admin loads AdminPanel.jsx.
2. Unauthenticated/non-admin users view the **Enterprise Admin Gateway** card.
3. Submitting admin@devstore.com / Admin123! unlocks the console.
4. Admin oversees real registered users, places stock updates, and manages active orders.

C. Order & Automated Shipping Lifecycle:

1. User adds products to cart and completes checkout at /checkout.
2. Order is created with status processing in order.json.
3. Admin view reflects new order dynamically.
4. After 24 hours, system automatically transitions delivery status to shipped.

5. Running & Deploying the System

- **Local Development:** Execute `npm run dev` from root to start both backend server (Port 5000) and frontend Vite server (Port 5173).
- **GitHub Repository:** Managed at <https://github.com/PRATHAMCHADDA/DevStore> on branch main.
- **Vercel Deployment:** Configured with Root Directory = client, Framework = Vite, and client/.npmrc (legacy-peer-deps=true) for React 19 compatibility.

Summary Note: DevStore is fully operational, thoroughly tested, and deployed to production. This document serves as an exhaustive technical handbook for learning and maintaining the project codebase.